

Lab10 - The Downfall of SHA1 and PRNG/CSPRNG

Part01 - The Downfall of SHA-1

We discussed in class the SHA-1 collision that was discovered in February 2017. The team at Google had planned to wait 90 days after their paper and announcement before they released their code for others to try it. However, the Internet beat them to it. Two days after the announcement a collider program was available online. You are going to use one of these programs to create a SHA-1 collision of two files. The collider program you will be using was written by Robert Xiao of Carnegie Mellon University. It is available on Github (<https://github.com/nneonneo/sha1collider.git>).

- a. Log into ISELab (iselab01.ece.iastate.edu) and onto your Ubuntu desktop.
- b. Download the code from repo. Be sure to include the ending period. Remember you will need to change permissions to write to these directories.
`scp -r repo@repo.homework.3310.com:~/lab10/ .`
- c. Install all needed packages. *You may need to apt update first.*
`sudo apt-get install libjpeg-progs`
`sudo apt-get -y install python3-pip`
`sudo python3 -m pip install Pillow argparse`
- d. Open LibreOffice Writer and write two different documents. One would be considered a "good" letter. For example something like a recommendation letter for someone. The other one would be considered an "evil" file. The example would be taking that recommendation letter and slamming that person ensuring he/she would not get the job (he/she is slothful, doesn't bathe, plays on his/her cell phone while at work, etc.) The only requirement on the two files is that they need to be the same number of pages.
- e. **Export** the LibreOffice files as pdf files. Save them in the `~/homework/lab10/part01/sha1collider` directory.
- f. Run the pdfs through the SHA-1 Collider program. From inside the `sha1collider` directory:
`python3 collide.py --progressive <filename1.pdf>`
`<filename2.pdf>`
- g. Open up the two out-filename.pdf files, and ensure that you can still read the message. *If the files are corrupted, try again with different messages.* **Upload the two out-filename.pdf files to Canvas.**
- h. Once you have ensured the files don't have any corruption, hash the two files using `sha1sum` and `sha512sum`. **Take a screenshot to include in your lab report.**
- i. **Explain why the SHA1 hashes are the same, but the SHA512 hashes are different. Please be specific about how the near collision/collision works.**
`sha1sum <out-filename1.pdf> <out-filename2.pdf>`
`sha512sum <out-filename1.pdf> <out-filename2.pdf>`
- j. How likely is this to be used to carry out an attack in the wild? Please give me a

clear example of when it could be used in the wild or when it would not work in the wild to support your argument. **Include both parts of your answer to these questions in your lab report.**

Part 02 - Deterministic PRNG - Mersenne Twister

When the assumption is made that random numbers are created correctly, bad things happen if they are not. In this part of the lab, you will be using Python's random module, which is based on the Mersenne Twister and `random.randint(0,100)` to produce user-visible numbers. The Mersenne Twister is a PRNG which are deterministic based upon the seed. The name comes from the use of a Mersenne prime modulus $2^{19937}-1$. The Twist in the name comes from using bitwise shifts, XORs, and a constant twist operation to create new state values. It is a type of feedback shift register, but has an extremely long period before repeating. It IS NOT cryptographically secure.

You will be given 4 observed outputs in order and you have to find the initial seed. The generator was seeded once at program start using a simple integer seed equal to the number of seconds since midnight on that day. That would be a value between 0 and 86399.

You will brute force the seed by trying all possible seeds in the range and comparing the produced sequence to the observed sequence. You stop the search when you find the first seed that produces a matching sequence. You will report the seed as a number and convert it to HH:MM:SS since midnight.

1. Start with the skeleton code `part02_seed_skel.py`.
2. The 4 observed values are in the list named `observed`
3. You will brute force the seed by trying all possible seeds in the range and comparing the output to the observed list of integers.
4. You will report the seed as a number and convert it to HH:MM:SS since midnight.
5. Take a screenshot of your output with both values clearly displayed and include it in your lab report.
6. Upload your code to Canvas.
7. Once you recover the seed, can you predict only future random numbers, or can you also reconstruct the past? (forward and backward predictability) Explain your reasoning.

Part 03 - CSPRNG

The Mersenne Twister is pseudorandom and deterministic. Anyone who knows the seed can reproduce the output. A cryptographically secure pseudorandom number generator (CSPRNG) uses unpredictable entropy and resists prediction even if part of the internal state is known.

In Python, we can use the modules `secrets` and `os.random`, which pull randomness from the operating system's cryptographically secure source. On Linux, that is `/dev/urandom`, and in Windows, it would be `CryptGenRandom`.

8. Run the demonstration code `part03_compare_prng_csprng_skel.py` 5 times.
9. Take a screenshot of the output each time. Include the 5 different screenshots in your lab report.
10. Compare the output of the three functions. Comment on the values being produced in each of the 5 iterations and whether each should be used in a cryptographic key.
11. Now run the demonstration code `part03_entropy_demo_skel.py`. This program runs each of the generators 1000 times and prints the number of unique values that have been created. Explain what the output means and why it is or isn't useful in a cryptographic application.
12. Include a screenshot of your final output in your lab report.
13. You used two cryptographically secure generators: `secrets` and `os.urandom`. If an attacker could see some of the values these functions produced, could they guess future values or recreate the internal state of either generator? Provide your reasoning.

Part 04 - CSPRNG for Keys and IVs

14. Run the demonstration code `part04_csprng_keys_skel.py`. This time, there is a loop in the program that will generate 5 keys and 5 IVs for you to compare..
15. Take a screenshot of the output to include in your lab report.
16. Looking at the 5 generated keys and IVs. Do any of them repeat? What could happen in AES encryption if the same IV were used with the same key?
17. Compare the 5 SHA-256 hashes of your 5 keys. Do the hashes differ a lot between the different keys? How much would the hashes differ if the keys only differed slightly (like by one bit)? What does this ensure?

Lab 10 Template

Part 01

1. **“outfile-Good text” pdf and “out-file Bad text” pdf (upload the files in canvas submission)**
(5 points)
2. **Screenshot of the hashes of the text PDFs colliding for SHA1 and differing with SHA512**
(5 points)
3. **Explain why the SHA1 hashes are the same for different files, but the SHA512 hashes are different. Please be specific about how the near-collision/collision works.**
(5 points)
4. **How likely is this to be used to carry out an attack in the wild? Give an example/support your claim**
(10 total points, 5 points likelihood, 5 points example)

Part 02

5. **Take a screenshot of your output with both values (found seed and time of day) clearly displayed and include it in your lab report.**
(5 points)
6. **Upload your code to Canvas.**
(5 points)
7. **Once you recover the seed, can you predict only future random numbers, or can you also reconstruct the past? (forward and backward predictability) Explain your reasoning.**
(10 total points, 5 points answer, 5 points explanation)

Part 03

8. **Include the 5 different screenshots in your lab report.**
(5 points)

- 9. Compare the output of the three functions. Comment on the values being produced in each of the 5 iterations and whether each should be used in a cryptographic key.**
(10 total points, 3.33 for each function)
- 10. Explain what the output from 1000 iterations of the generator means and why it is or isn't useful in a cryptographic application. Include a screenshot of your final output in your lab report.**
(5 points)
- 11. If an attacker could see some of the values from secrets and os.urandom, could they guess future values or recreate the internal state of either generator? Provide your reasoning.**
(10 total points, 5 points answer, 5 points reasoning)

Part 04

- 12. Screenshot of the output to include in your lab report.**
(5 points)
- 13. Do any of the 5 generated keys and IVs repeat? What could happen in AES encryption if the same IV were used with the same key?**
(10 total points, 5 points answer, 5 points reasoning)
- 14. Do the hashes differ a lot between the different keys? How much would the hashes differ if the keys only differed slightly (like by one bit)? What does this ensure?**
(10 total points, 5 points answer, 5 points reasoning)